

Media Optimization: Web Cheat Sheet

Baseline 2026-10-07. Native HTML images, BlurHash placeholders, Next.js, React, Angular and video players. Quick reference only: verify against the official docs and the full pages.

RESPONSIVE IMAGE

```
<picture>
  <source type="image/avif" srcset="w320.avif 320w, w640.avif 640w" sizes="(min-width:60em) 20vw, 50vw">
  <source type="image/webp" srcset="w320.webp 320w, w640.webp 640w" sizes="...">
  
</picture>
```

- srcset lists files, sizes says how wide the slot is (without it: 100vw assumed).
- width + height reserve space: no layout shift (CLS).
- AVIF first, WebP second, JPEG as the fallback.
- 1x/2x descriptors only for fixed-size images.

LOADING RULES

- loading="lazy" on everything below the first screen.
- **Never** lazy on the LCP image: use fetchpriority="high" (+ preload link with imagesrcset).
- decoding="async" on almost every image.
- fetchpriority="low" for carousel slides.
- Video: poster, preload="none" (lists) or metadata, playsinline, muted for autoplay.
- Audit: dimensions, sizes matches CSS, LCP loaded early, CLS near 0.

BLURHASH IN THE BROWSER

```
import { decode, isBlurhashValid } from 'blurhash';
const px = decode(hash, 32, 32, punch); // Uint8ClampedArray RGBA
ctx.putImageData(new ImageData(px, 32, 32), 0, 0);
canvas.toDataURL();
// data URL
```

- Decode at 20-32 px, stretch with CSS; set aspect-ratio on the box.
- Fade the image in on load; check img.complete; respect prefers-reduced-motion.
- punch: contrast. The reference decoder does punch | 1 (fractions truncate).
- No JS? Decode on the server and inline a 8-16 px data URL.
- react-blurhash 0.3.0: stable, quiet. unlazy 2.x: BlurHash/ThumbHash, Vue, Nuxt, SSR.
- Lifecycle: empty box → placeholder → fetch → decoded → cross-fade.

NEXT.JS (16)

```
<Image src={url} alt width height sizes="(min-width:60em) 20vw, 50vw" placeholder="blur" blurDataURL={dataUrl} />
<Image ... fill sizes="..." style={{objectFit:'cover'}} />
<Image ... preload /> // priority is deprecated
```

- Remote images need remotePatterns and a manual blurDataURL (decode the BlurHash on the server).
- images.formats: ['image/avif', 'image/webp'] (default WebP only). No cache invalidation: version the URL.
- Custom loader (receives src, width, quality) → CDN URL (Cloudinary, imgix, your variants); then Next does not optimise.
- preload or loading="eager" / fetchPriority="high" for LCP; do not combine.

ANGULAR NGOPTIMIZEDIMAGE

```
<img ngSrc="b81f/cover.jpg" width="640" height="960" sizes="50vw" [placeholder]="dataUrl" priority>
<img ngSrc="..." fill sizes="...">
```

- Lazy by default; priority = LCP image (fetchpriority + preload in SSR).
- Loaders: provideImgixLoader, provideCloudinaryLoader, provideImageKitLoader, provideCloudflareLoader, provideNetlifyLoader, or IMAGE_LOADER.
- placeholder: data URL (about 4 KB or less), blurred by default.
- ng-blurhash, ngx-blurhash, angular-blurhash: unmaintained. Decode with blurhash and guard with isPlatformBrowser.

REACT LIBRARIES

Option	Status	Use
native lazy + srcset	browser	default
react-lazyload 3.2.1	low activity	non-image content
react-image 4.1.0	no release since 2023	fallback chains
react-lazy-load-image-component 1.6.3	check repo	blur effect
IntersectionObserver hook	browser	custom triggers

VIDEO PLAYERS

Player	Formats	Notes
native video	MP4, WebM; HLS in Safari	no ABR elsewhere
hls.js	HLS	MSE; capLevelToPlayerSize
dash.js	DASH	reference player
Shaka Player	HLS + DASH	DRM, offline, UI
Video.js	HLS + DASH	skin, plugins

```
if (video.canPlayType('application/vnd.apple.mpegurl')) video.src = url;
else if (Hls.isSupported()) { const h = new Hls(); h.loadSource(url); h.attachMedia(video); }
```

- MSE needed; iOS: ManagedMediaSource in current versions.
- CORS for the player origin; signed cookies for private video; destroy players on unmount.

IMAGE CDN LOADERS

- Cloudinary: .../image/upload/f_auto,q_auto,w_WI DTH/ID
- imgix: ?auto=format&w=WIDTH&q=75
- Restrict allowed sizes or sign URLs: an open resizer is a cost risk.
- Prefer versioned URLs to purges.